Low-pass and High-pass Filtering of Sentinel-2 Imagery over the Aravalli Ridge

Which filtering technique suits which remote sensing application?

Group 1

Pranay Chakraborty (MS22041), Sangeeta Kumari (PH25084), Aparna Manoj (MS24070), Robin (MS23138), Sangam (MS23222)

EES403: Advanced Remote Sensing and GIS
Instructors: Dr. Chandrakant Ojha and Dr. Meenakshi Devi
Indian Institute of Science Education and Research, Mohali

[Sangeeta — 0:15 — 0:15]
Good morning. We are Group 1, and our project compares low-pass and high-pass filtering on Sentinel-2 imagery over the Aravalli ridge between Gurugram and Faridabad. The course brief asked us to implement both kinds of filter and decide which is most suitable for remote sensing. I will introduce the problem, Sangam will describe the data, Pranay will explain the methods, Aparna and Robin will present the results, and I will close with the recommendation.

Contents

1. Introduction — spatial frequency, aim
2. Study Area and Data — scene, reference map
3. Methods — filters, experiments
4. Results I — images, ringing, E1
5. Results II — E2 to E5
6. Discussion and Conclusions — decision matrix

Presenter

Sangeeta
Sangam
Pranay
Aparna
Robin
Sangeeta

The talk follows the report: Chapters 1 to 6, one section each.

[Sangeeta — 0:15 — 0:30]
The talk follows the chapters of our report. First the idea of spatial frequency and our aim, then the study area and data, then the methods, then the results in two parts, and finally the decision matrix and our recommendation.

Section 1: Introduction

Presented by **Sangeeta**

1. Introduction

2. Study Area and Data

3. Methods

4. Results I: Filtered Images, Ringing and Noise

5. Results II: Classification, Edges and Cost

6. Discussion and Conclusions

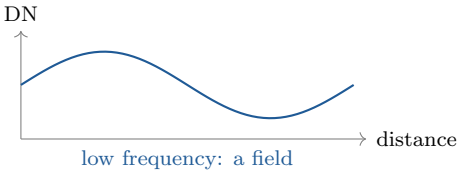
Spatial frequency is how often brightness changes per unit distance

Low spatial frequency: grey values change gradually
fields, water, broad slopes

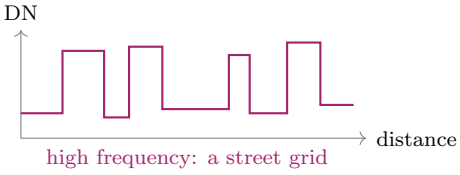
High spatial frequency: grey values change abruptly
roads, roofs, ridge crests, field boundaries, noise

Every image = broad pattern (low) + edges and texture (high).
A filter keeps one part and suppresses the other.

Definition after Lillesand et al. (2015).



low frequency: a field



high frequency: a street grid

[Sangeeta — 0:30 — 1:00]

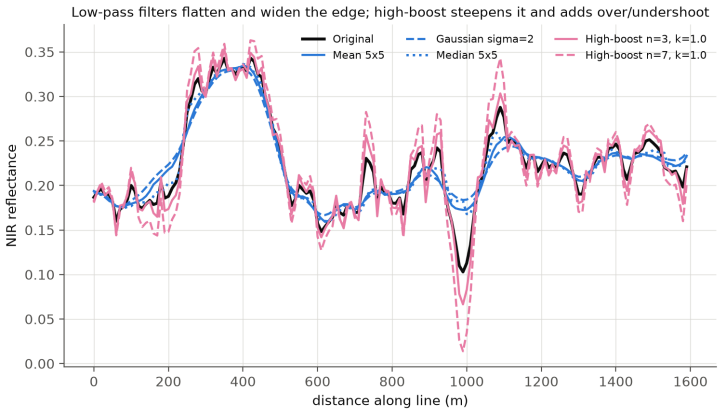
A satellite image is a grid of numbers. Spatial frequency is the number of changes in brightness per unit distance. Over a field or a lake the values change slowly; that is low frequency. Across a street grid or a ridge crest they jump every few pixels; that is high frequency. Any image can be thought of as a broad pattern plus a layer of edges and texture, and a filter lets us keep one and suppress the other.

Low-pass flattens edges; high-pass and high-boost sharpen them

Low-pass (LPF)
averages neighbours
⇒ smoothing, less noise

High-pass (HPF)
HPF = original − LPF
⇒ only edges remain

High-boost
original + k·HPF
⇒ sharper, colours kept



NIR reflectance along a 1.6 km line across a cleared patch on the ridge (report Fig. 1.1).

Both can be computed in the *spatial* domain (kernels) or the *frequency* domain (FFT masks).

[Sangeeta — 0:35 — 1:35]

A low-pass filter replaces each pixel by an average of its neighbours, so the image becomes smoother. A high-pass filter keeps only the abrupt changes; the classic remote sensing form is the original minus a smoothed copy. Adding a fraction of that edge layer back gives a high-boost image. The plot is real data along one line on the ridge. The black line is the original. The blue low-pass curves are flatter, and the pink high-boost curves are steeper and overshoot at every edge. Both families can be computed in two ways, which Pranay will explain.

Aim: decide which filter is most suitable, application by application

Smoothing and edge enhancement do opposite jobs, so we split the brief into five questions:

Experiment	Question
E1	Which filter removes noise best?
E2	Which filter helps automatic land-cover classification most?
E3	Which filter makes edges, boundaries and lineaments clearest?
E4	How much does each filter change the reflectance values?
E5	Is the spatial or the frequency route faster?

The answers are combined in a **decision matrix**: one best filter per application.

[Sangeeta — 0:25 — 2:00]

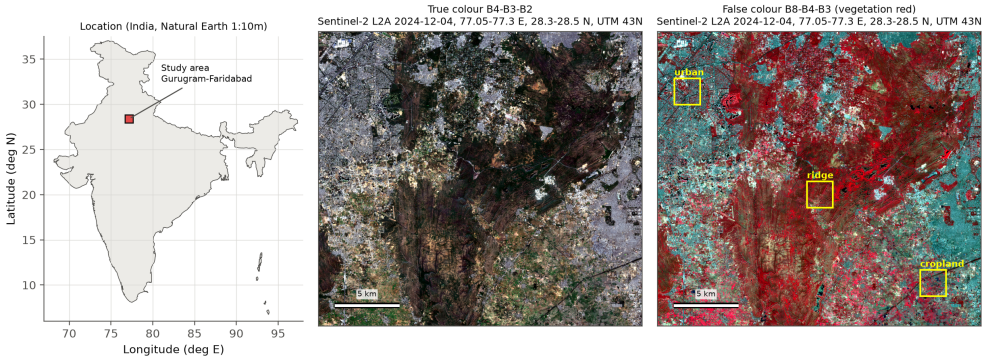
Since smoothing and sharpening are meant for different jobs, asking for one best filter would not make sense. So we split the question into five, each answered by one experiment: noise removal, classification, edges, change to the spectra, and speed. At the end the five answers are combined in a decision matrix. Sangam will now describe the area and the data.

Section 2: Study Area and Data

Presented by **Sangam**

1. Introduction
2. Study Area and Data
3. Methods
4. Results I: Filtered Images, Ringing and Noise
5. Results II: Classification, Edges and Cost
6. Discussion and Conclusions

The Aravalli ridge holds ridges, city and farmland in one scene



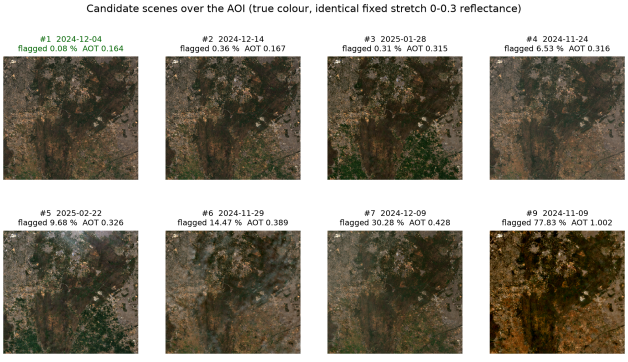
Left: location. Centre: true colour. Right: false colour (NIR, red, green), vegetation red. Yellow boxes: ridge, urban and cropland windows.

Area: 77.05–77.30°E, 28.30–28.50°N, 25 × 23 km
Low frequency: crop parcels in the south-east
High frequency: folded quartzite ridges, Gurugram streets
Also: old quarries, a few ponds

[Sangam — 0:40 — 2:40]

Our study area is a 25 by 23 kilometre box over the Aravalli ridge between Gurugram and Faridabad. We chose it because one scene holds very different textures. The folded quartzite ridges have long parallel linear features, and Gurugram has a dense street grid; both are high frequency and test the high-pass filters. The crop parcels in the south-east are smooth and test the low-pass filters. The three yellow boxes are two kilometre windows that we use for all zoomed comparisons later.

Scenes were ranked by cloud and haze inside the study area



Candidate scenes over the study area, same colour stretch; flagged share and mean AOT above each.

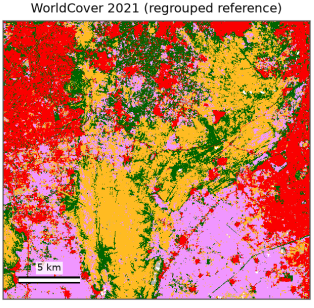
- Sentinel-2 Level-2A (Drusch et al., 2012), Google Earth Engine
- Nov 2024 – Feb 2025, scene cloud $< 5\%$: 11 scenes
- Flagged pixels *inside the box*: 0.08% to 97.2%
- Ranked by scene classification, s2cloudless and aerosol optical thickness (AOT)

Chosen scene	2024-12-04, tile 43RGM
Flagged pixels	0.08%, AOT 0.164
Bands	B2, B3, B4, B8 at 10 m
Grid	2491 × 2261 px, UTM 43N
Reflectance	DN / 10000, clipped to [0, 1]

[Sangam — 0:50 — 3:30]

We used Sentinel-2 Level-2A surface reflectance from Google Earth Engine, with the four 10 metre bands: blue, green, red and near-infrared. Winter is usually the clearest season over Delhi, so we searched November 2024 to February 2025 for scenes under five percent cloud; 11 qualified. But the scene-wide cloud figure was misleading: inside our box the share of cloudy, hazy or missing pixels ranged from 0.08 to 97.2 percent, because winter smog is not counted as cloud. So the program scored every scene inside the box only and picked the cleanest one, from 2024-12-04, with 0.08 percent flagged pixels. We used one real scene, not a composite, because compositing is itself a kind of smoothing.

ESA WorldCover 2021 provides the land-cover reference



WorldCover v200 (Zanaga et al., 2022), 10 m, regridded to the Sentinel-2 grid (nearest neighbour), classes merged:

Class	Share of area
Trees	19.9%
Shrub/Grass	28.7%
Cropland	24.6%
Built-up	25.3%
Bare/Mining	1.2%
Water (dropped, $< 0.5\%$)	0.19%

WorldCover is itself a classifier output: a *reference map*, not ground truth.

[Sangam — 0:30 — 4:00]

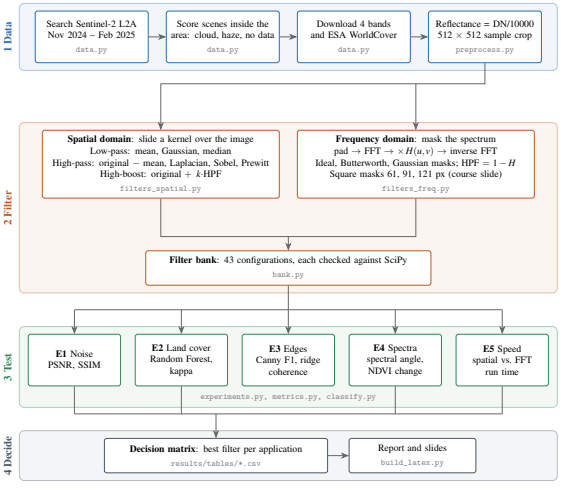
To judge classification and boundaries we needed a land-cover map. We used ESA WorldCover 2021 at 10 metres, put on the same pixel grid as our scene, and merged its classes into six groups. Water covered only 0.19 percent of the area, too little to train on, so it was dropped, leaving five classes. One caution: WorldCover is itself produced by a classifier, so it is a reference, not ground truth. Pranay will now explain how the filters and experiments work.

Section 3: Methods

Presented by **Pranay**

1. Introduction
2. Study Area and Data
3. Methods
4. Results I: Filtered Images, Ringing and Noise
5. Results II: Classification, Edges and Cost
6. Discussion and Conclusions

Workflow: data, filter, test, decide



- Python package `rsfilter`; grey text in each box names the module
- All filters written in NumPy, per band, on float32 reflectance
- Unit tests against `scipy.ndimage`: max. difference $< 10^{-5}$
- Mirror (reflect) padding at the image border
- One command, `python scripts/run_all.py`, repeats everything; seed 42

[Pranay — 0:30 — 4:30]

The work runs in four stages: data, filtering, testing and the decision. Every step is a function in our Python package. We wrote all filters ourselves in NumPy and used SciPy only as a reference inside unit tests; the largest disagreement is below ten to the minus five. At the border we mirror the image, so no artificial black frame enters the result. The random seed is fixed at 42 and one command reproduces every number in the report and on these slides.

Spatial domain: each output pixel is a weighted sum of its neighbours

$$g(x,y) \;=\; \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)\; f(x-s,\; y-t)$$

- f: input band, w: kernel of size $(2a+1) \times (2b+1)$, g: output
- Convolution flips the kernel by 180°; irrelevant for our symmetric kernels
- **Shift and accumulate**: pad once, then for every kernel cell (s,t)
  $g \mathrel{+}= w(s,t)\cdot$ (padded image shifted by (s,t))
  $\Rightarrow$ loop over $(2a+1)(2b+1)$ cells, not over millions of pixels
- Cost per pixel $\propto$ number of kernel cells $= n^2$ for an $n \times n$ kernel

Gonzalez and Woods (2018), Ch. 3.

[Pranay — 0:40 — 5:10]

In the spatial domain, every output pixel g is a weighted sum of the input pixels f around it, with the weights held in a kernel w. Formally this is a convolution, which rotates the kernel by 180 degrees, but all our kernels are symmetric so that makes no difference. Instead of looping over pixels, our code loops over the kernel cells: for each cell it shifts the whole padded image, multiplies by the weight and adds it to the output. This is memory-light and easy to verify. Note the cost grows with n squared; that matters for the run-time experiment.

Spatial kernels: low-pass averages, high-pass differences

Low-pass

Mean: $w_{\text{mean},3} = \frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$, $n = 3, 5, 7, 9$

Gaussian: $w(s,t) \propto \exp\!\left(-\frac{s^2+t^2}{2\sigma^2}\right)$
side $2\lceil 3\sigma \rceil + 1$, $\sum w = 1$, $\sigma = 1, 2, 3$

Median ($n = 3, 5$): median of the window; non-linear, ignores single extreme values

High-pass

$\text{HPF}_n = f - \text{mean}_{n\times n}(f)$ (Lillesand et al., 2015)

$w_{\text{Lap},4} = \begin{bmatrix}0 & 1 & 0\\ 1 & -4 & 1\\ 0 & 1 & 0\end{bmatrix}$ $w_{\text{Sobel},x} = \begin{bmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{bmatrix}$

Gradient magnitude $\sqrt{G_x^2 + G_y^2}$ (G_y: transposed kernel); Prewitt uses weights 1, 1, 1

High-boost: enhanced $= f + k \cdot \text{HPF}_n$, $k = 0.5, 1, 1.5$

[Pranay — 0:50 — 6:00]

On the left, the low-pass kernels. The mean filter gives equal weights. The Gaussian gives nearby pixels more weight; its size is two times the ceiling of three sigma plus one, and the weights sum to one so brightness is preserved. The median is not a weighted sum at all: it takes the median of the window, so a single spike is simply ignored. On the right, the high-pass side. The remote sensing HPF is the image minus its mean filter. The Laplacian is a second derivative and responds to any change in slope. Sobel and Prewitt estimate the first derivative in x and y and we take the gradient magnitude. High-boost adds k times the HPF back to the image.

Frequency domain: filtering becomes a multiplication of the spectrum

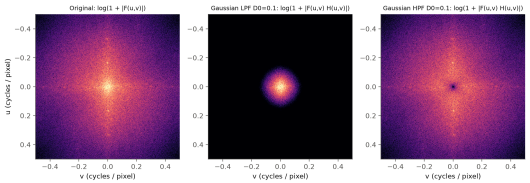
$$g = \mathcal{F}^{-1}\big\{H(u,v) \cdot \mathcal{F}\{f\}\big\}$$

Pipeline per band: mirror-pad 64 px → FFT → shift centre → $\times H$ → shift back → inverse FFT → real part → crop.

D: distance from spectrum centre, $D = 1$ at Nyquist; cutoff $D_0 = 0.1, 0.2, 0.3$

Ideal:	$H = 1$ if $D \le D_0$, else 0
Butterworth ($n = 2$) :	$H = 1/\big(1 + (D/D_0)^4\big)$
Gaussian:	$H = \exp\big(-D^2/2D_0^2\big)$
High-pass:	$H_{\mathrm{HP}} = 1 - H$

Gaussian mask ≡ spatial Gaussian with $\sigma = 1/(\pi D_0)$ px



Log-magnitude spectra of the NIR crop: original, Gaussian LPF and HPF ($D_0 = 0.1$).

A sharp cut in $H \Rightarrow$ a sinc-like kernel with negative side lobes ⇒ ripples (*ringing*, Gibbs, ≈ 9% overshoot).

[Pranay — 0:55 — 6:55]

In the frequency domain the convolution theorem turns filtering into a multiplication: Fourier transform, multiply by a mask H, transform back. We mirror-pad first because the FFT treats the image as periodic, and without padding the left edge would leak into the right. D is the distance from the centre of the spectrum, scaled so that D equals one at the Nyquist frequency. The ideal mask is a hard cut, Butterworth is a smooth roll-off, Gaussian is smoother still, and every high-pass mask is one minus the low-pass one. A Gaussian mask with cutoff D0 is the same filter as a spatial Gaussian with sigma one over pi D0, which gives us a check between the two domains. Finally, a hard cut in frequency corresponds to a sinc-shaped kernel in space, whose negative side lobes cause ripples next to edges; theory predicts about nine percent overshoot.

Filter bank: 43 configurations compared

Family	Filter	Parameters
Low-pass, spatial	Mean; Gaussian; Median	$n = 3, 5, 7, 9$; $\sigma = 1, 2, 3$; $n = 3, 5$
Low-pass, frequency	Ideal, Butterworth, Gaussian	$D_0 = 0.1, 0.2, 0.3$
High-pass, spatial	Original $-$ mean; Laplacian; Sobel, Prewitt	$n = 3, 5, 7$; 4-, 8-neighbour
High-pass, frequency	Ideal, Butterworth, Gaussian	$1 - H$, $D_0 = 0.1, 0.2, 0.3$
High-boost	$f + k \cdot \mathrm{HPF}_n$	$n = 3, 5, 7$; $k = 0.5, 1.0, 1.5$

Plus the course-slide example: square ideal low-pass masks of 61, 91 and 121 px on a 512×512 crop.

[Pranay — 0:15 — 7:10]

Together these give 43 configurations, covering both families in both domains. We also reproduced the example in the course slides with square ideal masks of 61, 91 and 121 pixels.

Five experiments, each with a measure that decides the winner

	Set-up	Measure
E1	Gaussian noise $\sigma = 0.02$ or 2% salt-and-pepper on a clean 512×512 crop; every LPF	PSNR $= 10 \log_{10}(1/\mathrm{MSE})$ dB; SSIM (Wang et al., 2004)
E2	Random Forest (Breiman, 2001), 200 trees, 2000 px/class; 4 bands + NDVI after filtering	$\kappa = (p_o - p_e)/(1 - p_e)$; accuracy; macro-F1; salt-and-pepper index
E3	NIR band	mean gradient; Canny (Canny, 1986) edge F1 vs WorldCover boundaries (2 px); ridge coherence
E4	all bands	SAM $= \arccos \frac{\mathbf{x}\cdot\mathbf{y}}{\lvert\mathbf{x}\rvert\lvert\mathbf{y}\rvert}$; $\lvert\Delta\mathrm{NDVI}\rvert$; RMSE
E5	one full band, 2261×2491 px	median of 5 runs, spatial vs FFT

[Pranay — 0:40 — 7:50]

Each experiment has one deciding measure. E1 uses PSNR, ten log ten of one over the mean squared error, since reflectance runs from zero to one; three decibels more means roughly half the error. E2 uses Cohen's kappa, which is the observed agreement p o corrected for the agreement p e expected by chance. E3 uses the F1 score of Canny edges against WorldCover class boundaries, and orientation coherence on the ridge. E4 uses the spectral angle between a pixel's four-band vector before and after filtering; zero degrees means the shape of the spectrum is unchanged. E5 is plain wall-clock time.

Two design choices that keep the evaluation honest

E2: spatially blocked split

Neighbouring pixels are near-copies, so a random pixel split inflates accuracy.

$\Rightarrow$ 128 $\times$ 128 px blocks, 70% train / 30% test (10 000 training, 1 700 310 test pixels)

E3: orientation coherence

Structure tensor $J = G_\rho * \begin{bmatrix} g_x^2 & g_x g_y \\ g_x g_y & g_y^2 \end{bmatrix}$, eigenvalues $\lambda_1 \geq \lambda_2$

$$C = \frac{\lambda_1 - \lambda_2}{\lambda_1 + \lambda_2} \in [0, 1]$$

1: parallel ridges; 0: speckle. 2 $\times$ 2 km ridge window (Bigün and Granlund, 1987).

No published lineament map exists here, so coherence stands in for lineament quality.

[Pranay — 0:30 — 8:20]

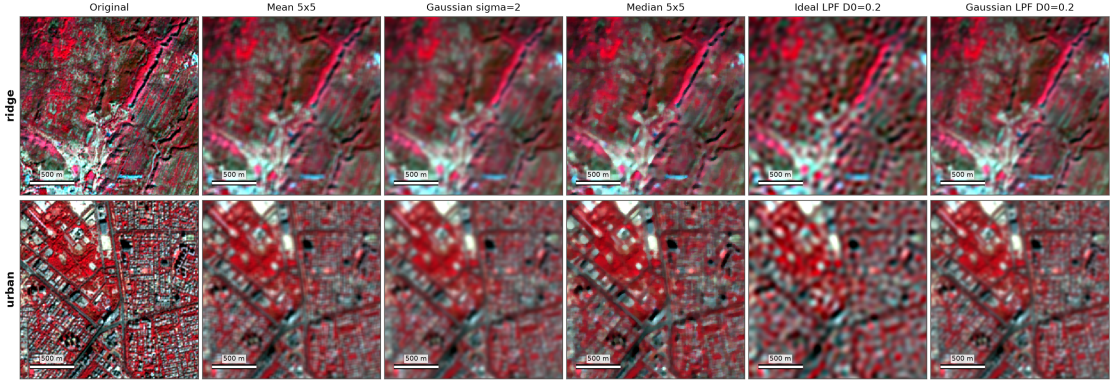
Two choices need explaining. First, neighbouring pixels are almost identical, so if test pixels are drawn at random each has a near-copy in the training set and accuracy looks better than it is. We cut the area into 128 pixel blocks and split the blocks 70 to 30. Second, there is no lineament map of this area, so we measure how linear the ridge window is. The structure tensor averages the outer product of the gradient; when one eigenvalue dominates, the gradients share one direction and coherence approaches one. Random speckle gives values near zero. Aparna will now show what the filters do.

Section 4: Results I: Filtered Images, Ringing and Noise

Presented by **Aparna**

1. Introduction
2. Study Area and Data
3. Methods
4. Results I: Filtered Images, Ringing and Noise
5. Results II: Classification, Edges and Cost
6. Discussion and Conclusions

Low-pass filters keep the broad ridge but blur streets and field edges



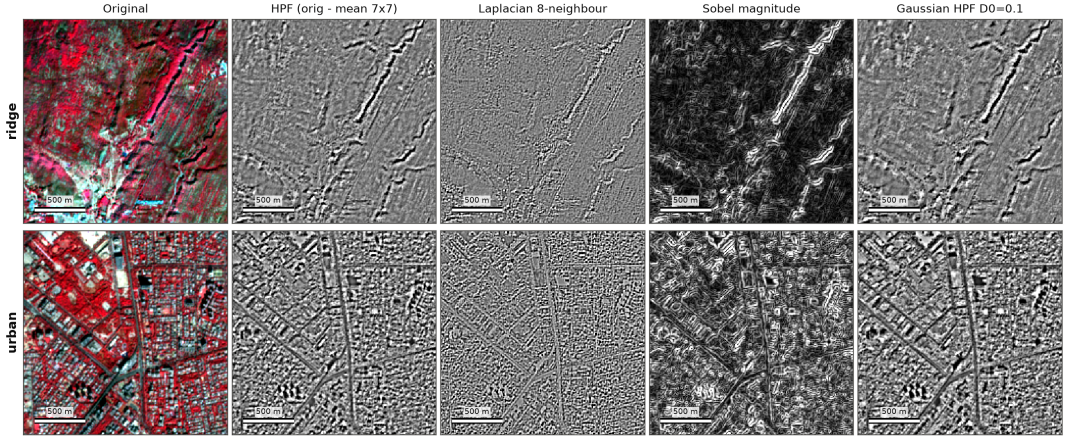
2 km windows, ridge (top) and Gurugram (bottom); false colour, same stretch in every panel.

The median keeps straight edges sharper than a mean or Gaussian of the same size.

[Aparna — 0:30 — 8:50]

These are two 2 kilometre windows, the ridge on top and Gurugram below, all with the same colour stretch. Every low-pass filter blurs the street grid and the field edges, while the broad shape of the ridge survives. The median keeps straight edges a little sharper, because it picks an existing value instead of averaging across the edge. The Gaussian frequency mask looks just like the spatial Gaussian, as Pranay predicted.

High-pass filters turn flat ground grey and keep only the edges



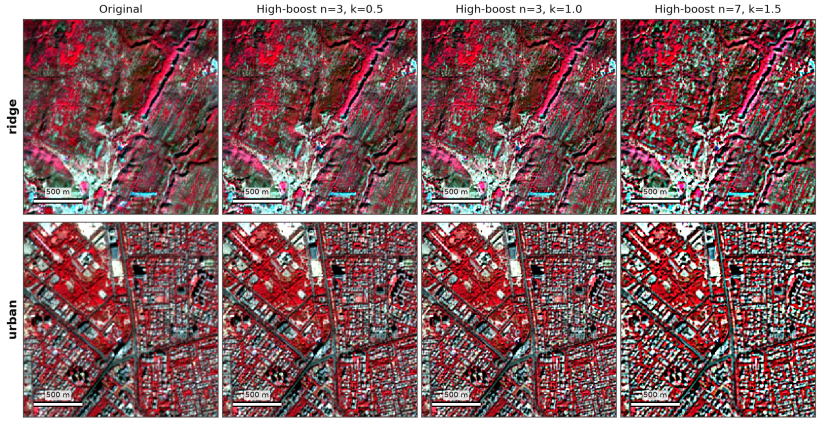
NIR band, same two windows.

7×7 HPF and Sobel trace the ridge crests; the Laplacian reacts mostly to fine speckle.

[Aparna — 0:25 — 9:15]

High-pass outputs are edge maps. Uniform ground becomes mid-grey and only the changes remain. The 7 by 7 HPF and the low-cutoff frequency HPF bring out the ridge crests, and Sobel shows edge strength in any direction. The Laplacian responds most to the finest speckle.

High-boost sharpens boundaries and keeps the colours

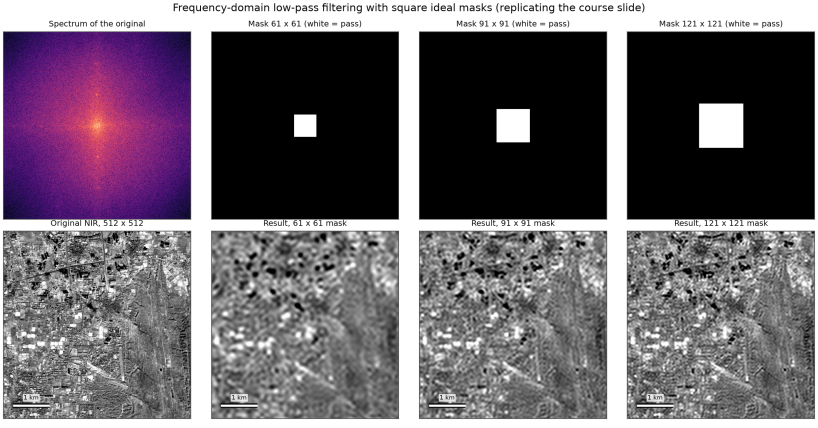


At $k = 1.5$, bright and dark halos appear along roads and ridge crests.

[Aparna — 0:20 — 9:35]

An analyst rarely looks at the bare edge layer, so here it is added back to the image. As k increases, boundaries get crisper while the colours stay the same. At k equal to 1.5 we start to see halos along roads and ridge crests, so a moderate k is better.

The course example reproduced: smaller square mask, blurrier image



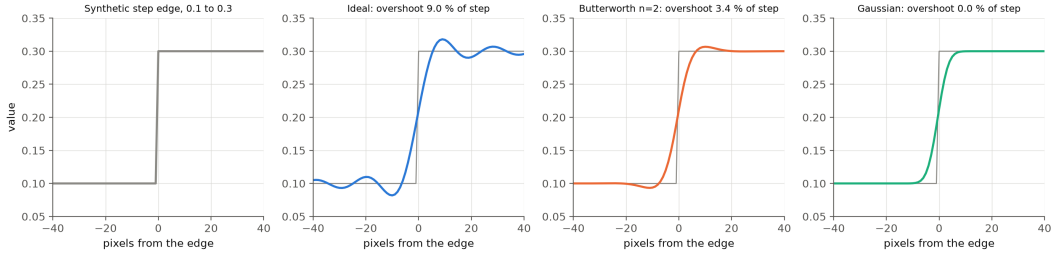
Square ideal low-pass masks of 61, 91 and 121 px (top) and the filtered NIR crop (bottom).

All three results show a faint ripple texture.

[Aparna — 0:20 — 9:55]

This reproduces the example from the course slides on our own crop. The top row shows the spectrum and the square masks; white means the frequency is kept. The smallest mask keeps the fewest frequencies and gives the blurriest image. All three results show a faint ripple texture, which the next slide explains.

A sharp frequency cut causes ringing; smooth masks do not



The same mask code applied to a clean synthetic step edge, $D_0 = 0.1$.

Mask ($D_0 = 0.1$)	Ideal	Butterworth	Gaussian
Overshoot, % of step height	**9.0**	3.4	0.0

Use Gaussian or Butterworth masks, never the ideal mask.

[Aparna — 0:30 — 10:25]

On real data the ripples are hidden by texture, so we passed a clean step edge through the same code. The ideal mask overshoots by 9.0 percent of the step height, which matches the roughly nine percent from theory. Butterworth overshoots by 3.4 percent, and Gaussian by 0.0. So whenever we filter in the frequency domain we should use a smooth mask.

E1: the best noise filter depends on the type of noise

	Gaussian, $\sigma = 0.02$		Salt-and-pepper, 2%	
Filter	PSNR (dB)	SSIM	PSNR (dB)	SSIM
Noisy input (no filter)	33.98	0.825	21.01	0.603
Mean 3×3	35.52	0.882	29.29	0.714
Mean 5×5	32.60	0.775	30.39	0.695
Gaussian $\sigma = 1$	**35.56**	**0.885**	30.27	0.748
Gaussian $\sigma = 2$	32.26	0.765	30.91	0.728
Median 3×3	35.19	0.872	**36.49**	**0.910**
Median 5×5	32.65	0.779	32.89	0.794
Ideal LPF $D_0 = 0.3$	33.05	0.799	29.65	0.676
Butterworth LPF $D_0 = 0.3$	34.33	0.848	30.71	0.746
Gaussian LPF $D_0 = 0.3$	35.22	0.876	30.48	0.749

Random noise: only light filters help; 13 of 18 made it worse.

Impulse noise: median 3×3 raises PSNR from 21.01 to 36.49 dB, 5.42 dB ahead of the best linear filter.

Median 3×3 for impulse noise; Gaussian $\sigma = 1$ or mean 3×3 for random noise.

[Aparna — 0:35 — 11:00]

For E1 we added noise to a clean crop and measured how close each filtered result came to the clean image; higher PSNR is better. With random noise the noisy image scored 33.98 decibels, and only the lightest filters improved on it; 13 of the 18 low-pass filters made it worse, because they removed more real detail than noise. With salt-and-pepper noise the 3 by 3 median was far ahead: a mean spreads each spike over its neighbours, while the median throws it away. Robin will present the remaining experiments.

Section 5: Results II: Classification, Edges and Cost

Presented by **Robin**

1. Introduction
2. Study Area and Data
3. Methods
4. Results I: Filtered Images, Ringing and Noise
5. Results II: Classification, Edges and Cost
6. Discussion and Conclusions

E2: low-pass inputs classify best, high-boost inputs worst

Input	Family	Accuracy	κ	Macro-F1	Salt-and-pepper
No filter	none	0.536	0.403	0.473	0.263
Gaussian $\sigma = 2$	LPF, spatial	0.613	0.500	0.539	**0.113**
Gaussian LPF $D_0 = 0.2$	LPF, frequency	**0.615**	**0.502**	**0.540**	0.118
High-boost $n = 3$, $k = 0.5$	high-boost	0.500	0.360	0.444	0.316
High-boost $n = 7$, $k = 1.5$	high-boost	0.431	0.275	0.383	0.378
Raw + HPF 7×7 layers	raw + HPF stack	0.598	0.481	0.528	0.178

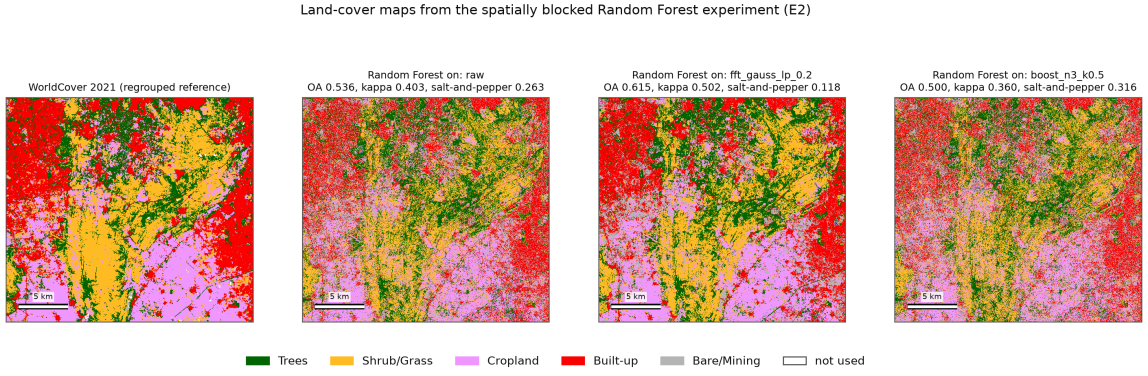
Held-out blocks. Best and worst of each family; bold = best (lowest for salt-and-pepper).

Smoothing removes within-class variation (roof types, shadows, mixed pixels) before classification.

[Robin — 0:35 — 11:35]

In E2 a Random Forest classified the area after each filter. On the raw bands kappa was 0.403. Every low-pass input did better; the best reached 0.502, and the spatial Gaussian was almost tied with it, which is expected because, as Pranay showed, the two are nearly the same filter. High-boost inputs did worse, at best 0.360. The reason is that at 10 metres one class contains many different surfaces, like roofs and shadows. Smoothing averages that variation out; sharpening makes it larger. Adding high-pass layers next to the raw bands helped a little, kappa 0.481, but less than smoothing.

E2: smoothing also halves the speckle in the classified map



Salt-and-pepper index: raw 0.263 → best low-pass 0.118.

[Robin — 0:20 — 11:55]

Here are the maps. From the left: the WorldCover reference, the map from the raw bands, from the best low-pass input, and from the best high-boost input. The raw and high-boost maps are speckled; the low-pass map is much cleaner. The share of pixels that disagree with their neighbours fell from 0.263 to 0.118.

E3: high-pass helps boundaries and ridge lineaments

Boundaries (Canny edge F1)

No filter	0.675
High-boost $n = 5$, $k = 1.5$	**0.742**
Gaussian $\sigma = 1$	0.479
Mean 5×5	0.247
Built-up only: no filter	0.543
Built-up only: best	0.545

Ridge lineaments (coherence)

No filter	0.510
Prewitt	**0.536**
Sobel	0.535
HPF 7×7	0.502
Laplacian (4-nb.)	0.426
Gaussian $\sigma = 3$	0.754*

*high only because smoothing erases speckle

Smoothing removes edges; high-pass filtering helps wherever the task is *where things change*.

[Robin — 0:40 — 12:35]

E3 asks which filter makes edges easiest to find. Edges detected on the original matched the WorldCover boundaries with an F1 of 0.675. High-boost raised this to 0.742, and smoothing lowered it, because fewer edges survive. For built-up boundaries alone the gain was negligible, and we report it as such. On the ridge, the gradient operators Prewitt and Sobel raised coherence slightly above the original. The Laplacian scored lowest because it picks up speckle. Strong smoothing scores higher still, but only because it erases the speckle along with the fine detail, so we do not count it as better lineament mapping.

E4: light filters keep the spectra; HPF layers are not reflectance

Filter	Spectral angle (°)	|ΔNDVI|
High-boost $n = 3$, $k = 0.5$	1.20	–
Median 3×3	2.22	0.033
Gaussian $\sigma = 1$	2.30	0.037
Sobel	18.90	–
HPF 7×7	88.76	–

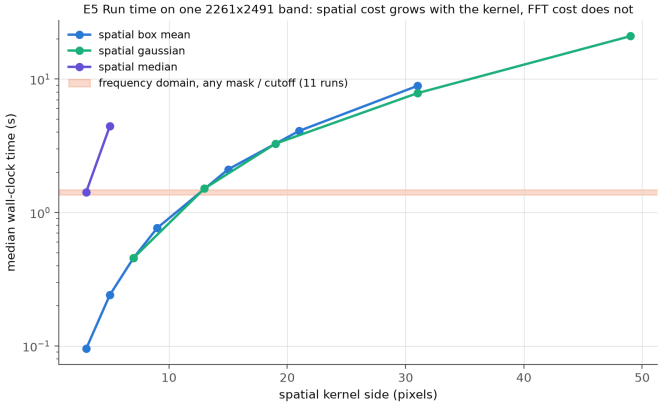
Mean spectral angle to the original image; 0° = spectrum shape unchanged.

Keep high-pass products as *extra layers* beside the bands, never in place of them.

[Robin — 0:25 — 13:00]

E4 measures what filtering costs. Light low-pass filters and mild high-boost change the spectrum by only one or two degrees. A pure high-pass layer is a different thing: its mean is near zero and its spectral angle to the original is close to 90 degrees, so it is not reflectance any more and an NDVI from it has no meaning. That is why high-pass products should be stored beside the bands, not instead of them.

E5: FFT cost is flat, convolution cost grows with kernel size



One full band, 2261×2491 px:

- Spatial mean 3×3: 0.10 s
- Spatial mean 31×31: 8.88 s
- Spatial Gaussian 49×49: 20.96 s
- Any FFT mask: 1.36–1.48 s

Break-even near a 13-px kernel

Small kernels: convolve. Large kernels or many cutoffs: use the FFT.

[Robin — 0:25 — 13:25]

Finally speed, on one full band. A 3 by 3 spatial filter took 0.10 seconds, but the time grows quickly with the kernel, up to 20.96 seconds for the largest. Every frequency-domain filter took about the same time, between 1.36 and 1.48 seconds, whatever the mask. The two lines cross near a kernel of 13 pixels. Since the filters that worked best are small, convolution is usually the practical choice. Sangeeta will bring the results together.

Section 6: Discussion and Conclusions

Presented by **Sangeeta**

1. Introduction

2. Study Area and Data

3. Methods

4. Results I: Filtered Images, Ringing and Noise

5. Results II: Classification, Edges and Cost

6. Discussion and Conclusions

Decision matrix: one best filter per application

Application	Best filter	Evidence (best vs. no filter)
Noise removal, random noise	Gaussian $\sigma = 1$	PSNR (dB): **35.56** vs 33.98
Noise removal, impulse noise	Median 3×3	PSNR (dB): **36.49** vs 21.01
Land-cover classification	Gaussian LPF $D_0 = 0.2$	Cohen's κ: **0.502** vs 0.403
Lineament mapping (ridges)	Prewitt	ridge coherence: **0.536** vs 0.510
Urban boundary delineation	High-boost $n = 3$, $k = 0.5$	built-up edge F1: **0.545** vs 0.543
Visual interpretation	High-boost $n = 3$, $k = 1.0$	NIR mean gradient: **0.0417** vs 0.0245

Generated by the analysis code from the experiment tables (`decision_matrix.csv`).

Low-pass when pixel values feed a calculation; **high-pass** when edges and structure are the point.

[Sangeeta — 0:35 — 14:00]

This table answers our five questions. It is produced by the code from the experiment results, with one measure per application. Some gains are large, like the median on impulse noise or the Gaussian for classification. Some are small, like urban boundaries and ridge coherence, and we present them as small. The pattern is consistent: low-pass filtering wins where pixel values feed a calculation, and high-pass filtering wins where the task is about where things change.

Recommendation

Quantitative analysis

classification, indices, change detection

⇒ light **Gaussian**, $\sigma \approx 1.5$–2 px (or $D_0 \approx 0.2$)
⇒ **median** 3×3 for impulse noise

Interpretation and mapping

visual analysis, boundaries, lineaments

⇒ **high-boost**, $k = 0.5$–1, for display
⇒ **Sobel / Prewitt** layer for lineaments

Avoid

the ideal frequency mask (ringing), and heavy smoothing (detail is lost before noise)

[Sangeeta — 0:30 — 14:30]

So our recommendation is split by purpose. For quantitative work, use light low-pass filtering: a small Gaussian, or a 3 by 3 median when there are dropped or saturated pixels. For interpretation and structural mapping, keep the original bands and add a high-pass product beside them: high-boost with moderate k for display, and a Sobel or Prewitt layer for lineaments. Avoid the ideal mask and avoid heavy smoothing.

Limitations and conclusions

Limitations

- WorldCover is a model output, not ground truth
- One winter scene, one area
- One block split: κ differences ≈ 0.01 are not a ranking
- Coherence is a proxy for lineament quality
- Bare/Mining (1.2%): F1 ≤ 0.14 for every input

Conclusions

- No single filter is "most suitable"; the two families do opposite jobs
- Smoothing: κ +0.099, half the map speckle
- Sharpening: best boundary match and sharpest image
- Spatial and FFT routes agree for smooth masks

Thank you. Questions?

Live demo: `python -m rsfilter demo` (offline, 512 × 512 sample)

[Sangeeta — 0:30 — 15:00]

Our results have limits. WorldCover is itself a model output; we used one winter scene and one area; and kappa differences of about 0.01 between the leading filters should not be read as a ranking. To conclude: there is no single most suitable filter, because the two families do opposite jobs. Smoothing gave the most accurate and cleanest land-cover map, and sharpening gave the best boundary match and the sharpest image. Thank you. We are happy to take questions, and we can run the offline demo if you would like to see it.

References I

Josef Bigün and Gösta H. Granlund. Optimal orientation detection of linear symmetry. In *Proceedings of the First International Conference on Computer Vision*, pages 433–438, London, 1987.

Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6): 679–698, 1986.

M. Drusch, U. Del Bello, S. Carlier, O. Colin, V. Fernandez, F. Gascon, B. Hoersch, C. Isola, P. Laberinti, P. Martimort, A. Meygret, F. Spoto, O. Sy, F. Marchese, and P. Bargellini. Sentinel-2: ESA's optical high-resolution mission for GMES operational services. *Remote Sensing of Environment*, 120:25–36, 2012.

European Space Agency. Sentinel-2 Level-2A Product Specification and Sen2Cor Algorithm Theoretical Basis Document. ESA Sentinel Online.

Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, New York, 4th edition, 2018.

John R. Jensen. *Introductory Digital Image Processing: A Remote Sensing Perspective*. Pearson, Glenview, IL, 4th edition, 2015.

Thomas M. Lillesand, Ralph W. Kiefer, and Jonathan W. Chipman. *Remote Sensing and Image Interpretation*. Wiley, Hoboken, NJ, 7th edition, 2015.

Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

Daniele Zanaga, Ruben Van De Kerchove, Dirk Daems, Wanda De Keersmaecker, Carsten Brockmann, Grit Kirches, Jan Wevers, Oliver Cartus, Maurizio Santoro, Steffen Fritz, et al. ESA WorldCover 10 m 2021 v200. Zenodo, 2022.